



Relatório Projeto I - verificação de cenários e determinação de área limpa por um robô aspirador

Maria Eduarda Teixeira Costa e Sofia Gazolla da Costa Silva

Departamento de Informática e Estatística - Universidade Federal de Santa Catarina

Estrutura de Dados

Professores: Alexandre Goncalves Silva e Aldo Von Wangenheim

Outubro 2025

1 Introdução

O Projeto I consistiu na implementação de duas atividades. A primeira foi a validação de um arquivo XML, na qual se verificava a correção do aninhamento e o fechamento das tags. A segunda atividade consistiu na implementação de um algoritmo que simula um robô aspirador cuja função era limpar as posições preenchidas pelo valor '1' em uma matriz binária.

Ambas as atividades fizeram o uso de estruturas de dados lineares estudadas na disciplina. Na primeira, fez-se o uso de pilhas: quando uma tag de abertura era detectada, ela era empilhada; ao encontrar uma tag de fechamento, seu nome deveria ser equivalente ao elemento no topo da pilha. Já na segunda atividade, foi utilizada a estrutura de fila para armazenar as coordenadas dos pontos (posições na matriz) onde o robô aspirador encontrou o dado '1'.

2 Desenvolvimento - Problema 1

2.1 Lógica da Função

Para a resolução da primeira atividade, que visava verificar o aninhamento e o fechamento correto das tags nos arquivos XML, foi implementada a função `verificaAninhamento()`. Para isso, utilizamos a estrutura de pilha, já desenvolvida em aulas anteriores.

O método consiste em percorrer todo o texto do XML, identificando o início de cada tag (`'<'`). Se o caractere seguinte for a barra (`'/'`), a tag é classificada como de fechamento. A lógica principal é a seguinte: se for detectada uma tag de abertura, seu nome é imediatamente empilhado (**push**). Por outro lado, se for uma tag de fechamento, o algoritmo verifica se há erros de aninhamento incorreto (o nome da tag de fechamento não coincide com o nome da tag no topo da pilha) ou se há um fechamento sem a devida abertura (pilha vazia). Se nenhum erro for encontrado, a tag no topo é desempilhada (**pop**). Ao final, a pilha deve estar vazia para que o arquivo XML seja válido.

2.2 Implementação

```
// Problema 1 : validação de arquivo XML
bool verificaAninhamento(string &texto){

    ArrayStack<string> pilha;
    size_t indice = 0;

    while(indice < texto.length()) {

        if (texto[indice] == '<') {
            indice++;

            string nome_da_tag = "";
            bool e_tag_de_fechamento = false;

            if (indice < texto.length() && texto[indice] == '/') {
                e_tag_de_fechamento = true;
                indice++;
            }

            while (indice < texto.length() && texto[indice] != '>') {
                nome_da_tag += texto[indice];
                indice++;
            }

            if (!e_tag_de_fechamento) {
                pilha.push(nome_da_tag);
            } else {
                if (pilha.empty()) {
                    return false;
                }
                if (pilha.top() != nome_da_tag) {
                    return false;
                }
                pilha.pop();
            }
        } else {
            indice++;
        }
    }

    return pilha.empty();
}
```

Figure 1: Função verificaAninhamento

2.3 Imagem

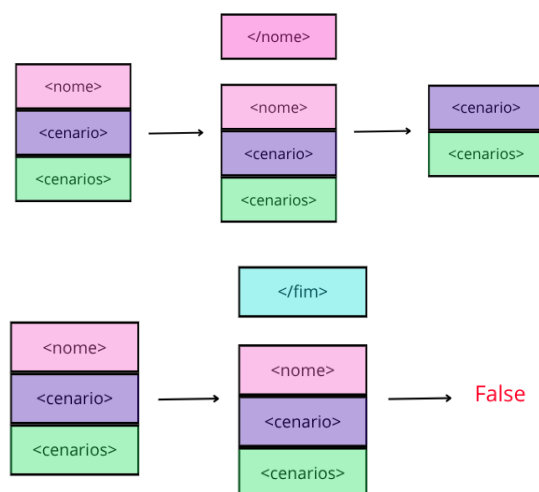


Figure 2: Ilustração do funcionamento da pilha de verificaAninhamento()

3 Desenvolvimento - Problema 2

3.1 Lógica da Função

O método `area_limpeza` implementa o algoritmo de Busca em Largura (BFS) para calcular o tamanho da área contínua acessível ao robô.

Nesta função, foi feito o uso de uma fila `ArrayQueue`, que recebe elementos do tipo `pair<size_t, size_t>` para armazenar as coordenadas (x e y) dos espaços a serem explorados. Além disso, foi criada Matriz de Visitados (`matriz_r`), inicializada com '0's, cuja função é registrar todas as posições já exploradas. O algoritmo é iniciado verificando se o ponto de partida do robô contém '1' (limpável); se sim, essa posição é marcada na `matriz_r`, enfileirada, e a contagem da área é iniciada.

O processo de exploração ocorre enquanto a fila não estiver vazia. A cada iteração, uma posição é retirada da fila e seus quatro vizinhos (`vizinhanca_4`) são analisados. Antes de qualquer verificação de limpeza, é realizado um teste para garantir que a próxima posição é válida e não está fora dos limites da matriz. Apenas se a posição for válida, contiver '1' na `matriz_entrada` e ainda não tiver sido visitada (valor '0' na `matriz_r`), ela é adicionada à fila, marcada como visitada, e somada ao total da área limpa.

3.2 Implementação

```
size_t area_limpeza(Cenario &cenario){
    string matriz_entrada = cenario.matriz;
    size_t x = cenario.x;
    size_t y = cenario.y;
    size_t altura = cenario.altura;
    size_t largura = cenario.largura;

    size_t area_limpeza = 0;

    vector<pair<int, int>> vizinhanca_4 = {{-1,0}, {1,0}, {0,-1}, {0,1}};

    ArrayQueue<pair<size_t, size_t>> fila(altura * largura);

    string matriz_r = "";
    for (int i = 0; i < matriz_entrada.length(); i++){
        matriz_r += '0';
    }

    if (matriz_entrada[x * largura + y] == '1'){
        matriz_r[x * largura + y] = '1';
    }

    while (!fila.empty()){
        pair<size_t, size_t> fila_auxiliar = fila.dequeue();

        for (int i = 0; i < 4; i++) {
            size_t x0 = fila_auxiliar.first + vizinhanca_4[i].first;
            size_t y0 = fila_auxiliar.second + vizinhanca_4[i].second;

            // Posicoes validas
            if (x0 < altura && x0 >= 0 && y0 < largura && y0 >= 0 ) {
                size_t posicao_valida = x0 * largura + y0;
                if (matriz_entrada[posicao_valida] == '1' && matriz_r[posicao_valida] == '0') {
                    fila.enqueue(make_pair(x0, y0));
                    matriz_r[posicao_valida] = '1';
                    area_limpeza++;
                }
            }
        }
    }

    return area_limpeza;
}
```

Figure 3: Função area_limpeza

3.3 Imagem

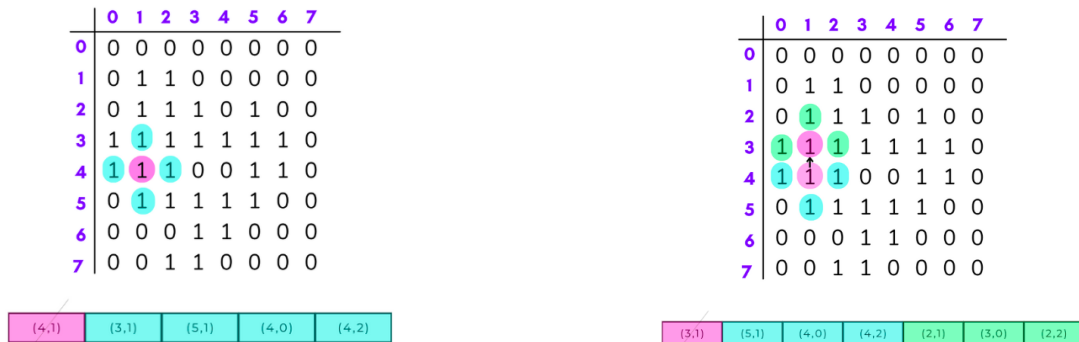


Figure 4: Ilustração do funcionamento da fila de area_limpeza()

4 Dificuldades

O primeiro desafio significativo do projeto foi garantir a leitura sequencial e correta dos múltiplos cenários a partir do arquivo XML. A dificuldade inicial estava na arquitetura da classe Cenario, cujo construtor não avançava o índice de leitura para consumir a tag de

fechamento `/cenario`. Isso impedia a iteração correta no `main`, resultando no travamento do programa ou na leitura incorreta dos dados. A solução adotada foi a criação da função `numero_de_matrizes`. Essa função permitiu pré-calcular o número exato de cenários a serem processados, garantindo assim o fluxo correto de leitura do arquivo.

```
// Conta <matriz>, para saber quantas matrizes teremos no nosso cenário
size_t numero_de_matrizes(const string &texto) {
    size_t contador = 0;
    size_t posicao = 0;

    while((posicao = texto.find("<matriz>", posicao)) != string::npos){
        contador++;
        posicao = posicao + 8;
    }
    return contador;
}
```

Figure 5: Função `numero_de_matrizes`

Em relação à segunda atividade, inicialmente, havíamos implementado uma estrutura utilizando structs e um `std::vector` que tínhamos chamado de `Matriz2D`, replicando a estrutura da matriz na memória. No entanto, o problema dos testes falharem na leitura dos cenários nos levou à percepção de que a representação da matriz no XML era, basicamente, uma longa string contendo apenas '0's e '1's. Foi então decidido abandonar o uso da `Matriz2D` e utilizar apenas a classe `std::string` para representar as matrizes de entrada e de visitados. Esta simplificação permitiu reduzir a complexidade do código.

References

- [1] INSTITUTO METRÓPOLE DIGITAL. *Material Didático – Aula 09: Descoberta de caminho – Parte 03 (Busca em Largura)*. MaterialPublic, UFRN. Disponível em: <https://materialpublic.imd.ufrn.br/curso/disciplina/5/69/9/4>. Acesso em: 14 out. 2025.
- [2] CPLUSPLUS.COM. *string::npos — cplusplus.com reference*. Disponível em: <https://cplusplus.com/reference/string/string/npos/>. Acesso em: 14 out. 2025.
- [3] GEEKSFORGEEEKS. *std::find in C++*. Disponível em: <https://www.geeksforgeeks.org/cpp/std-find-in-cpp/>. Acesso em: 14 out. 2025.

- [4] SCALER TOPICS. *Vector Pair in C++*. Disponível em: <https://www.scaler.com/topics/vector-pair-in-cpp/>. Acesso em: 14 out. 2025.